

Practical Report 8

SE3082 - Parallel Computing

IT24103532 – H.S.S Udayaratna

Exercise 1

In Exercise 1, a basic MPI program ('HelloMPI.c') is implemented to verify the MPI runtime environment and demonstrate the Single Program, Multiple Data (SPMD) execution paradigm.

The program initializes the parallel environment using 'MPI_Init' and assigns a unique identifier (rank) to each process via 'MPI_Comm_rank'. Additionally, 'MPI_Get_processor_name' retrieves the hostname of the core or compute node executing each process. Conditional logic (if (rank == 0)) is utilized to distinguish the master process from the worker processes, allowing rank 0 to print a primary greeting while other ranks output standard process notifications. Finally, 'MPI_Finalize' properly releases allocated resources and terminates the MPI session. This exercise confirms successful multi-process execution, process rank differentiation, and hardware resource identification.

```
sihan@Sihan:/mnt/c/Users/sihan/OneDrive/Desktop/PC lab 8 IT24103532/SE4060-MPI-Part1$ mpic++ -o job1 HelloMPI.c
sihan@Sihan:/mnt/c/Users/sihan/OneDrive/Desktop/PC lab 8 IT24103532/SE4060-MPI-Part1$ mpirun --oversubscribe -np 2 ./job1
Just a normal process From rank 1 machine Sihan
Hello World! From rank 0 machine Sihan
sihan@Sihan:/mnt/c/Users/sihan/OneDrive/Desktop/PC lab 8 IT24103532/SE4060-MPI-Part1$
```

Exercise 2

In Exercise 2, a parallel summation program (ex02_sum.c) is implemented to compute the sum of integers from 1 to 10,000,000 across multiple MPI processes.

The total workload is evenly partitioned into contiguous chunks based on the process rank and total communicator size (MPI_Comm_size), ensuring that each process independently calculates a local partial sum. Using blocking point-to-point communication, worker processes (rank > 0) transmit their partial results to the master process (rank 0) using MPI_Send. The master process receives each contribution using MPI_Recv, accumulates them into the final total sum, and measures total execution time using MPI_Wtime(). This exercise demonstrates domain decomposition, workload balancing, point-to-point communication, and parallel execution timing.

```
sihan@Sihan:/mnt/c/Users/sihan/OneDrive/Desktop/PC lab 8 IT24103532/SE4060-MPI-Part1$ mpic++ -o ex02_sum ex02_sum.c
sihan@Sihan:/mnt/c/Users/sihan/OneDrive/Desktop/PC lab 8 IT24103532/SE4060-MPI-Part1$ mpirun --oversubscribe -np 4 ./ex02_sum
Sum 1..10000000 = 50000005000000
Processes: 4 Time: 0.003575 seconds
```

Exercise 3

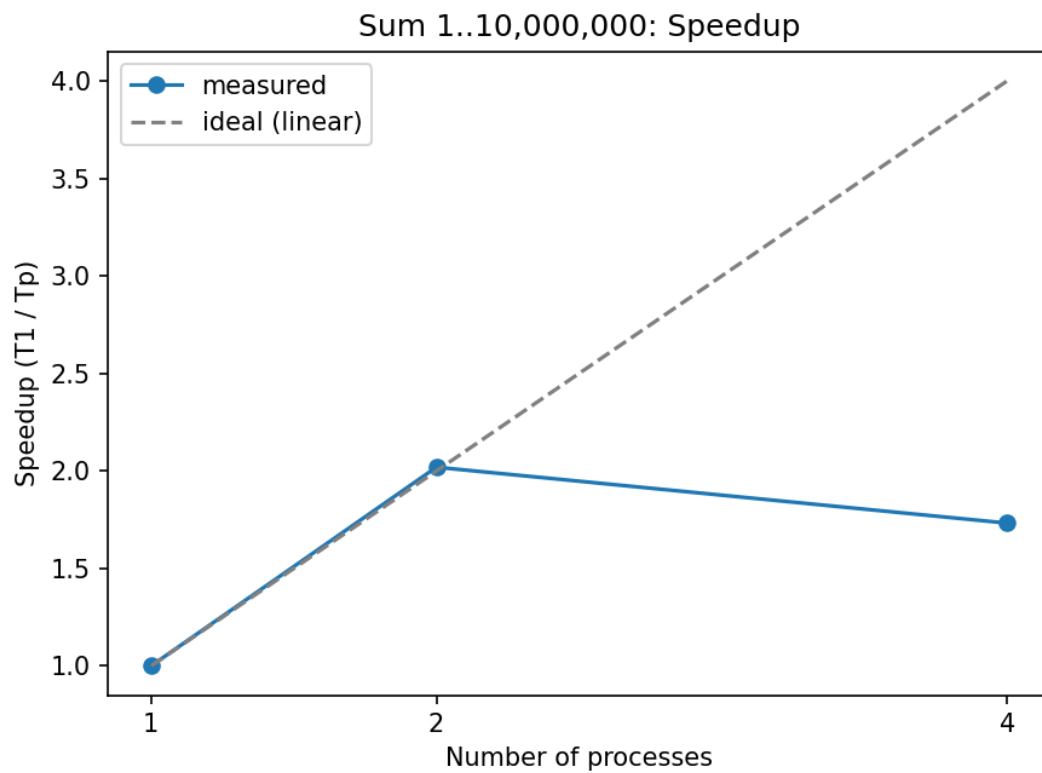
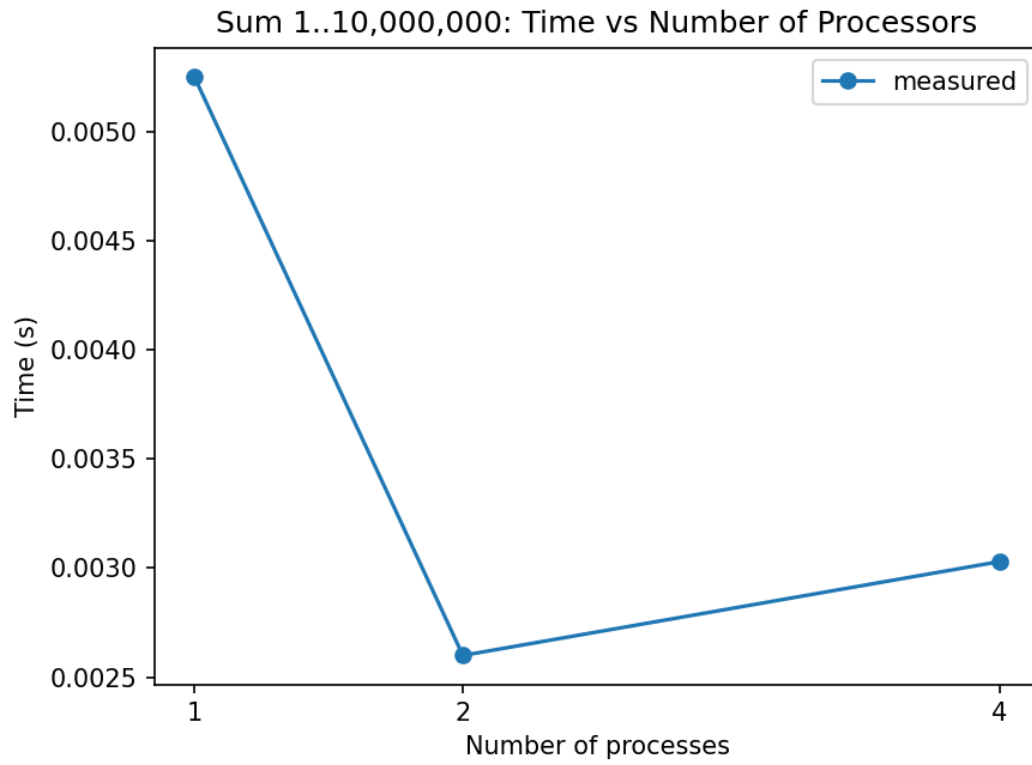
In Exercise 3, the Monte Carlo method is implemented in parallel (`ex03\_pi\_montecarlo.c`) to estimate the mathematical constant Pi (π) using 10,000,000 random points.

The total number of trials is divided equally among the available MPI processes. To ensure independent, non-overlapping random sequences, each process initializes its random number generator with a unique seed derived from the system time and process rank. Each process then generates coordinates (x, y) in the range $[0, 1]$ and determines how many points fall within the unit circle ($x^2 + y^2 \leq 1.0$). Worker processes transmit their local hit counts to rank 0 via `MPI\_Send`. The master process receives and aggregates these counts using `MPI\_Recv`, computes the final approximation ($\pi \approx 4 \times \frac{\text{total_hits}}{\text{total_points}}$), and measures execution runtime using `MPI\_Wtime()`. This exercise highlights parallel stochastic simulations and distributed reduction logic.

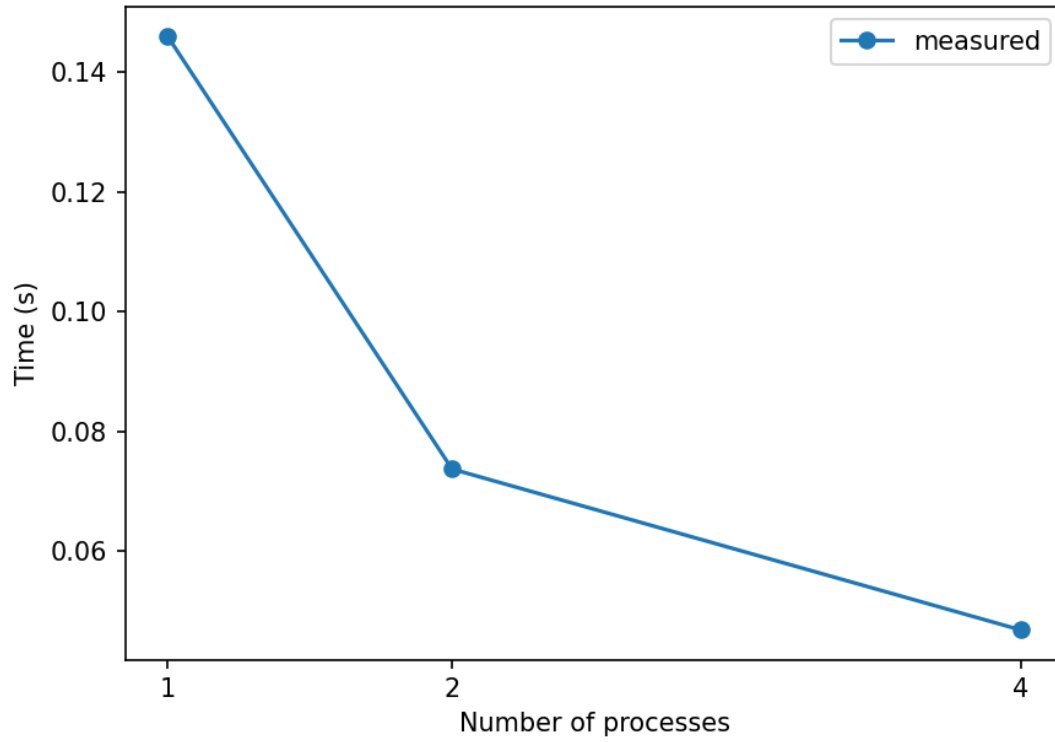
```
sihan@Sihan:/mnt/c/Users/sihan/OneDrive/Desktop/PC lab 8 IT24103532/SE4060-MPI-Part1$ mpic++ -o ex03_pi_montecarlo ex03_pi_monte
carlo.c
mpirun --oversubscribe -np 4 ./ex03_pi_montecarlo
Estimated Pi = 3.142182
Processes: 4 Time: 0.097666 seconds
sihan@Sihan:/mnt/c/Users/sihan/OneDrive/Desktop/PC lab 8 IT24103532/SE4060-MPI-Part1$
```

Exercise 4

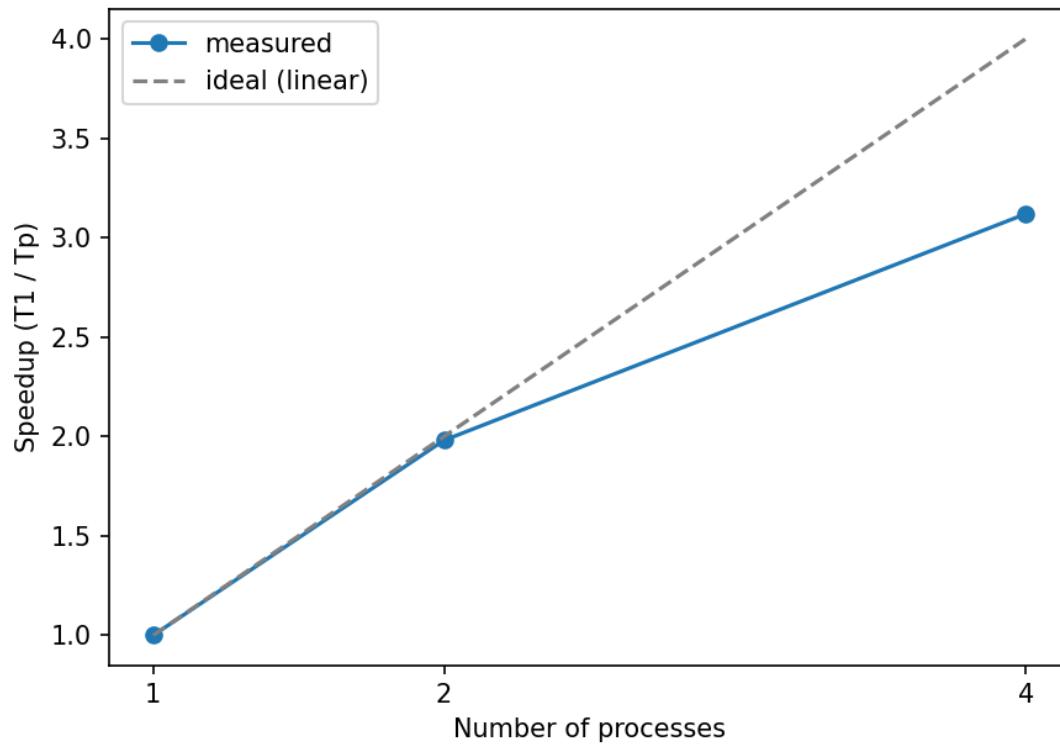
Both programs were executed using 1, 2, and 4 processes. Each test was run five times, and the fastest execution time was recorded. Speedup was calculated using the formula T_1 / T_p . The results show that speedup improves when increasing from 1 to 2 processes, but shows little improvement beyond that. This is because each t3.micro instance provides 2 vCPUs that share a single physical core, so the two-node cluster effectively offers only about two physical cores. As more processes are added, they must share the same underlying hardware resources, which limits further performance gains.



Monte Carlo Pi, 10,000,000 darts: Time vs Number of Processors



Monte Carlo Pi, 10,000,000 darts: Speedup



Exercise 5 (Part 1): Source/Destination Mismatch & Deadlock Demonstration

In this exercise (ex05_1_mismatch.cc), a communication mismatch is deliberately introduced to demonstrate blocking behavior and deadlock in MPI. Rank 0 sends a message targeted at Rank 1. However, Rank 1 specifies an explicit source rank of 2 in its MPI_Recv call. Because Rank 2 never initiates a send operation, Rank 1 blocks indefinitely waiting for matching incoming data. This demonstrates that standard point-to-point receive calls require an exact match for source rank (or the wildcard MPI_ANY_SOURCE), otherwise the receiving process will deadlock.

```
sihan@Sihan:/mnt/c/Users/sihan/OneDrive/Desktop/PC lab 8 IT24103532/SE4060-MPI-Part1$ mpic++ -o ex05_1_mismatch ex05_1_mismatch.cc
sihan@Sihan:/mnt/c/Users/sihan/OneDrive/Desktop/PC lab 8 IT24103532/SE4060-MPI-Part1$ mpirun --oversubscribe -np 3 ./ex05_1_mismatch
Rank 0: sending 42 to rank 1...
Rank 0: MPI_Send returned.
Rank 1: waiting for a message FROM RANK 2 (mismatch - rank 0 is the real sender)
Rank 2: has nothing to send, does nothing.
```

Exercise 5 (Part 2)

In Exercise 5 (Part 2), the program (ex05_2_bsend.cc) demonstrates buffered point-to-point communication using **Buffered Send (MPI_Bsend)**.

A dedicated memory buffer is dynamically allocated and attached to the MPI environment using MPI_Buffer_attach, with sufficient capacity to hold multiple outgoing messages plus MPI_BSEND_OVERHEAD. In a loop, Rank 0 sends sequential integer values to Rank 1 using MPI_Bsend. Because MPI_Bsend copies data directly into the attached buffer before returning, the sender is immediately unblocked and can overwrite the message variable without waiting for the receiver (Rank 1) to call MPI_Recv. Rank 1 subsequently retrieves all transmitted messages in order. Finally, the buffer is safely released using MPI_Buffer_detach and deallocated with free(). This exercise highlights how buffered communication decouples sender-receiver timing to prevent transmission delays and deadlocks.

```
sihan@Sihan:/mnt/c/Users/sihan/OneDrive/Desktop/PC lab 8 IT24103532/SE4060-MPI-Part1$ mpic++ -o ex05_2_bsend ex05_2_bsend.cc
sihan@Sihan:/mnt/c/Users/sihan/OneDrive/Desktop/PC lab 8 IT24103532/SE4060-MPI-Part1$ mpirun --oversubscribe -np 2 ./ex05_2_bsend
Rank 0 sent: 0
Rank 0 sent: 10
Rank 0 sent: 20
Rank 0 sent: 30
Rank 0 sent: 40
Rank 1 received: 0
Rank 1 received: 10
Rank 1 received: 20
Rank 1 received: 30
Rank 1 received: 40
sihan@Sihan:/mnt/c/Users/sihan/OneDrive/Desktop/PC lab 8 IT24103532/SE4060-MPI-Part1$
```

Exercise 6: Asynchronous Message Reception Using MPI_ANY_SOURCE

In Exercise 6 (ex06_pi_anysource.c), the Monte Carlo Pi calculation from Exercise 3 is refactored so that the master process (rank 0) receives incoming partial results using the wildcard source identifier MPI_ANY_SOURCE. In standard ordered receives, the master process must wait for each worker sequentially in rank order (\$1, 2, \dots, size - 1\$), which can cause idle waiting if higher-ranked processes finish earlier than lower-ranked ones. By utilizing MPI_ANY_SOURCE, the master process accepts incoming messages in a first-come, first-served manner regardless of which worker completes its computation first. The master tracks the total number of expected messages in a loop, aggregates the hits, and calculates the final estimate of π . This eliminates idle pipeline stalls and improves dynamic load resilience.

```
sihan@Sihan:/mnt/c/Users/sihan/OneDrive/Desktop/PC lab 8 IT24103532/SE4060-MPI-Part1$ mpic++ -o ex06_pi_anysource ex06_pi_anysource.c
sihan@Sihan:/mnt/c/Users/sihan/OneDrive/Desktop/PC lab 8 IT24103532/SE4060-MPI-Part1$ mpirun --oversubscribe -np 4 ./ex06_pi_anysource
Estimated Pi = 3.141798
Processes: 4 Time: 0.087152 seconds
sihan@Sihan:/mnt/c/Users/sihan/OneDrive/Desktop/PC lab 8 IT24103532/SE4060-MPI-Part1$
```

Exercise 7: Combining Buffered Send (MPI_Bsend) with Wildcard Receive (MPI_ANY_SOURCE)

In Exercise 7 (ex07_pi_any_bsend.c), both concepts from Exercise 5 (Part 2) and Exercise 6 are integrated into the Monte Carlo Pi estimation application.

Each worker process dynamically allocates a local memory buffer and attaches it via MPI_Buffer_attach before dispatching its hit count with MPI_Bsend. Because buffered send transfers the payload immediately into the attached application buffer, worker processes return immediately from the send call and proceed to clean up and finalize their execution without waiting on the master. Concurrently, the master process (rank 0) uses non-deterministic wildcard receives (MPI_ANY_SOURCE) to collect partial counts as soon as any worker completes. Once all messages are aggregated, the master computes the final Pi estimation and prints the total runtime. Finally, workers detach and free their buffers using MPI_Buffer_detach and free(). This architecture fully decouples sender and receiver timelines, maximizing parallel throughput and preventing communication bottlenecks.

```
sihan@Sihan:/mnt/c/Users/sihan/OneDrive/Desktop/PC lab 8 IT24103532/SE4060-MPI-Part1$ mpic++ -o ex07_pi_any_bsend ex07_pi_any_bsend.c
sihan@Sihan:/mnt/c/Users/sihan/OneDrive/Desktop/PC lab 8 IT24103532/SE4060-MPI-Part1$ mpirun --oversubscribe -np 4 ./ex07_pi_any_bsend
Estimated Pi = 3.141092
Processes: 4 Time: 0.095300 seconds
sihan@Sihan:/mnt/c/Users/sihan/OneDrive/Desktop/PC lab 8 IT24103532/SE4060-MPI-Part1$
```